Replay Engineering and Policy

How to define, test and interpret deterministic evidence

Before assessing determinism, identify the thing being assessed.

Grounded DI LLC | Mark S. Weinstein | 2 November 2026 | v1.1

Record note

This book explains a selection of Grounded DI's public replay work and the questions it can help a reader ask. Its purpose is to make exact reproducibility assessable: identify what is fixed, what is compared, how it is checked and what the evidence can support.

The exhibits report local project results from defined tests. Their outcomes are attributed to pinned public records. During the v1.1 audit, the project reran the V4 suite and selected source files for the water calculation; the other exhibits were not rerun. Separate project checkers are not unaffiliated certification. The scenarios described as synthetic contain fictional records and do not establish real-world events.

Exact identity can be valuable when a reviewer needs to compare a specified record, calculation or controlled action. It does not by itself establish that an input was true, a result was fair, a receiver acted, or a model will always generate identical responses. The framework in this book is an assessment aid, not a statement of law or a compliance certificate.

Reading route: begin with the assessment question, then use the case studies and source ledger to follow a claim back to its public record.

What is being assessed?

The same words can describe different tests. A claim becomes useful when it identifies both the object and the boundary at which it is observed.

MODEL RESPONSE | Compare generated responses under a specified model, input and execution environment. The surrounding demonstrations do not establish universal identity of model generation.

DECISION | Reapply declared rules to specified inputs and compare the decision. Reproducibility does not establish that the rules or inputs are substantively correct.

HISTORY | Check a recorded sequence of state transitions. Replaying a saved order does not imply a new concurrent run will have the same schedule.

ARTIFACT | Compare a particular file or canonical representation by bytes and, where appropriate, a digest. A match identifies the compared representation; it does not establish how the file was created.

AUTHORIZATION AND EXECUTION | Test whether a particular action is allowed or refused under stated controls. A simulated local gate does not automatically govern other processes or systems.

RECEIVER AND EVENT | Inspect a receiver record or its current state. A record's evidentiary value depends on its origin, binding and trust boundary. Current state alone may not distinguish different histories.

These are overlapping questions, not a mandatory stack. A single case may test several, while leaving others untested.

Make an exact claim testable

Begin with the proposition: what should be identical, or what action must be refused? Define the relevant input data, initial state, source versions, policy, software and execution conditions. Identify the output or event to observe and the comparison method before evaluating a result.

A direct byte comparison answers whether the compared byte sequences match. A cryptographic digest is a compact identity check with collision assumptions; it is not an explanation of the content or its provenance. Canonicalization specifies how a representation is encoded so that irrelevant differences do not accidentally alter a comparison. The chosen canonical boundary should be disclosed.

Record actual process facts separately where needed. A process identifier, timestamp or concurrent winner can differ while a defined canonical record remains equal. Conversely, identical projections can hide a meaningful difference in real history.

Test changed inputs and failure paths as well as a healthy baseline. A legitimate change can properly produce a different result. Missing or conflicting evidence may justify HOLD or REVIEW rather than a manufactured answer.

The question for an evaluator is not simply whether something was labeled deterministic. It is whether the defined property was tested under a declared contract and whether the reported evidence supports that specific claim.

The selected record

The eight exhibits below are an editorial selection of public records, not a count of all distinct technologies, deployments or benchmark families in the registry. Several are related versions or replays. The source ledger links each item to a pinned record.

S1 ARTIFACT AND PACKAGING | Seven-Hash closure separates six computational outputs from repackaging a historical delivery ZIP.

S2 LEGAL EVIDENCE | A fictional legal chain preserves separate authority, testimony, product and sufficiency stages.

S3 SCIENCE | A source-bound water calculation checks a defined equation and its intermediate records.

S4 AUTHORIZATION | Cooperative local processes compete for one export authorization; crash cases retain uncertainty.

S5 RECOVERY | A local write succeeds before confirmation is lost; recovery assesses retained records and bytes.

S6 HISTORICAL LIMIT | Matching canonical outputs do not distinguish a gate write from a planted matching file.

S7 RECONCILIATION | Fictional shipment and receiver records support one reconciliation and four reviews.

S8 LEGAL OUTPUT | A local closed-world evaluator compares legal artifacts and refuses a specified invalid export.

Together they illustrate multiple kinds of check. A successful check at one boundary is not automatically a finding about every other boundary.

S1 Artifact identity and packaging

Question: can the specified computation and the final package be reproduced, and are those the same claim?

The public closure record reports two previously executed fresh computational builds matching six reference boundaries: CSV, NPZ, PNG, manifest, prime mask and Merkle root. For boundary seven, two packaging processes reconstructed the unchanged historical 116-member payload and matched the original ZIP byte for byte. That is canonical repackaging of a frozen payload, not fresh generation of all its contents.

The first final-recheck package included seven unwanted Python cache entries and did not match. A clean extraction and corrected sequence produced the reported matches. Preserving this failure makes the packaging boundary clearer.

Implication: a comparison is useful when it names the bytes and the process that generated or repackaged them. Limit: the cited closure record identifies the evidence archives but states they were not hosted in that repository. It does not establish cross-platform reproducibility or universal model determinism. [S1]

S2 A legal evidence chain

Question: can a reviewer trace which evidence supports each stage without collapsing distinct legal propositions?

The public synthetic matter links authority and proposition binding, testimony tracing, product and exposure analysis, and litigation-sufficiency synthesis. The recorded two fresh executions matched the canonical stage and final records. The final disposition was INSUFFICIENT; the record identifies unresolved evidence, including unquantified dose, a missing causation opinion and a testimony contradiction.

Controls changed source-linked facts and introduced downstream alterations. A separately implemented project evaluator checked the chain without importing stage logic. Its shared fixture and specifications mean the paths are related project checks, not unaffiliated review.

Implication: a repeatable chain can preserve the distinction between identifying a product and establishing material, exposure, dose, causation or liability. Limit: the authorities, facts and matter are fictional; the result is not legal advice or a finding about a real case. [S2]

S3 A source-bound calculation

Question: can a specified scientific calculation be traced from an identified source through intermediate values to an output?

The record uses an attributed project transcription of NIST liquid-water heat-capacity coefficients. For a synthetic one mole heated from 298 K to 300 K, it reports an integrated heat of approximately 150.727449 J. The model assumes liquid water with no heat loss or phase change; the displayed source coefficients and table value have finite precision.

Two recorded runs matched seven defined boundaries. The seventh commits to the first six. A separate project checker uses a factorized algebraic form and shares the source record, assumptions, host and underlying libraries. Nine registered controls passed in the source record, including changed coefficients, invalid bounds and a wrong output with a recalculated commitment.

Implication: exact arithmetic and source binding make a calculation reviewable under its declared assumptions. Limit: the computation is not a physical experiment; the transcription is not an authenticated source response, and the comparison does not validate every scientific assumption. [S3]

S4 Authorization and interruption

Question: can a local authorization prevent a second export attempt when requests compete or a process stops?

In the public cooperative-store test, twenty processes competed for one authorization. The record reports one completed export, nineteen completed-state acknowledgments and one export-open attempt. It also reports fourteen scenarios and three tested process-kill points. After reservation without a completion record, recovery retains UNCERTAIN rather than returning the authorization to unused.

The separate project checker replays the recorded transition order and checks bindings and history without importing the gate or workers; it shares codec primitives. A new concurrent run can have a different winner, process IDs and order. Its safety outcome is the relevant comparison, not identical scheduling.

Implication: a controlled action and its recovery state can be tested as a defined local property. Limit: the claim is at most one export-creation attempt per authorization within the cooperative store. It allows uncertainty or noncompletion and is not distributed exactly-once delivery. [S4]

S5 Lost confirmation

Question: what can recovery conclude when a write occurs but its acknowledgment is lost?

In the V3 baseline, the local process recorded a pre-write record, wrote and flushed a fictional candidate file, then exited before returning success. A fresh retry found matching local evidence and returned ALREADY_PRESENT without a second write attempt. The public record reports two matching baseline executions and twelve passed scenarios.

Changes to candidate or destination bytes, missing or inconsistent records, and a different request identity were among the scenarios. A separate project checker recalculated the baseline using the request, policy and observed files, sharing the project environment and primitives.

Implication: an interrupted operation needs a declared recovery rule and evidence boundary. Limit: the pre-write record was unsigned and the local process and directory were trusted. Matching records and bytes do not by themselves prove who wrote the file or establish its historical cause. [S5]

S6 When identical records hide different histories

Question: can matching canonical outputs tell whether the gate actually wrote the destination file?

In the V4 planted-file fixture, the process generated a valid pre-write test receipt and stopped before the gate wrote. The fixture separately planted matching candidate bytes. Recovery returned ALREADY_PRESENT, with zero gate write attempts and one fixture write. The planted case and lost-confirmation baseline produced the same seven canonical outputs. The source reports fifteen passed scenarios.

The project verification record lists the compared output hashes and the distinct observed write counts. Both HMAC keys were public test fixtures: they do not provide real-world authentication. The separate checker shared test keys, inputs, host and libraries.

Implication: exact identity is meaningful for the artifacts that were compared, but those artifacts did not distinguish the two historical causes. A receipt, a signature or an additional hash cannot by itself prove unique historical execution. Evidence for that stronger claim requires a suitable trust and persistence boundary. [S6]

S7 Reconciliation and missing evidence

Question: when shipment and receiver records disagree, which quantities can be reconciled?

In a fictional five-shipment fixture, one record set matched the order, shipment, destination, receiver quantity and inventory change, reconciling ten supply cases. Four others went to review: a short receipt, wrong destination, missing receiver record and duplicate records. Duplicate entries were not summed into an assumed second delivery.

The source reports two fresh processes matching seven frozen canonical artifacts and a successful clean-extraction audit. Seven registered controls passed, including altered intermediates and a wrong final record with a recomputed commitment. A separate project checker shared the fixture, specification, host and serialization primitives.

Implication: reviewable reconciliation can preserve uncertainty instead of inventing a missing receipt or resolving a discrepancy by assumption. Limit: these are fictional unauthenticated records; a matching record in the fixture is not evidence of a physical delivery. [S7]

S8 A selective legal outcome

Question: can a closed-world evaluator produce the declared selective result and refuse a fixture-defined invalid hearsay input before export?

The published record reports three fresh executions of a newly implemented local evaluator with five corresponding canonical artifacts matching by bytes and SHA-256. Its fixture-defined invalid hearsay control was refused before export, with zero export-file opens and zero bytes written. The evaluator's declared result changed the punitive-damages outcome while retaining other specified outcomes in the fictional benchmark.

The separate project verifier did not import evaluator logic. The original evidence supplied reference artifacts rather than original executable engine source. The published record expressly does not claim original-engine execution, native product reproduction or actual-jurisdiction legal validation.

Implication: separate output comparisons and a pre-export refusal test answer different questions and can be presented together. Limit: this is a finite project-local demonstration, not a determination about a real case or universal model behavior. [S8]

Assessment guide

Use this checklist to turn a broad reproducibility or control claim into a reviewable test. It is a starting point; the right criteria depend on the system and the consequence of the action.

1. OBJECT AND PROPERTY | Which component, output, action or history is covered? State the precise claim.

2. INPUT AND STATE | What source material, request, policy, configuration and initial state are relevant? What is excluded?

3. EXECUTION CONDITIONS | Which code, versions, runtime, dependencies and trust boundaries are assumed? What may change?

4. OBSERVATION | Which bytes, decisions, state changes or receiver observations will be examined? What remains outside the observation?

5. METHOD | Is the check direct comparison, recalculation, replay of recorded history, a fresh execution, or an action-boundary test? Is there a separate checker and what does it share?

6. CHANGED CONDITIONS | What should happen when valid inputs change, or evidence is missing, stale, inconsistent or malformed?

7. FAILURE AND CONTROL | Which negative controls, interruption points and expected HOLD or REVIEW states were tested? Preserve failures and corrections.

8. CLAIM LIMIT | What does a passing test establish, and what cannot be inferred about correctness, authentication, historical execution or deployment?

For policy and procurement, this approach translates a general goal into observable obligations while allowing different systems and use cases to specify the right boundary. It does not prescribe one test for every system.

Illustrative procurement requirement

This example is an assessment starting point for a negotiated requirement. It is not a statement of law, a universal standard or a claim that an existing product satisfies it.

For the specified export gate, the supplier shall identify the code version, policy, canonical encoding, complete test request and initial authorization state. With the same frozen inputs and stated runtime, two fresh local executions shall produce the same canonical decision record, verified by direct byte comparison and SHA-256 against agreed expected bytes or hashes.

When the destination differs from the authorized destination, the gate shall refuse the request before opening the export file. When required authorization evidence is missing or inconsistent, the gate shall hold or refuse the operation under an agreed failure rule. A changed legitimate request requires a separate declared test and expected outcome; it shall not be treated as a defective input merely because it differs from the frozen fixture.

The test record shall retain the contract, versions, inputs, process outcomes, canonical outputs, comparison results and negative-control observations. A review shall identify shared dependencies and any actions outside the gate. The result does not itself establish correctness of source facts, protection from other writers or historical execution at a remote receiver.

Glossary

BOUNDARY | The specified point or representation at which an input, result or action is observed or checked.

CANONICAL REPRESENTATION | A defined encoding of a record for comparison. Its scope determines what differences are preserved or omitted.

FRESH EXECUTION | A new run of specified code under recorded conditions. It is distinct from rechecking a saved history or repackaging prior files.

HASH OR DIGEST | A fixed-size value derived from bytes and useful for integrity comparisons. A digest alone does not prove truth or origin.

NEGATIVE CONTROL | A deliberately altered or invalid case used to test whether a check responds as expected.

REPLAY | Re-execution, recalculation or checking of a recorded history under a specified contract. The record must say which sense is intended.

RECEIPT | A record asserting something about an operation. Its evidentiary value depends on what it binds, who or what issued it, and the applicable trust boundary.

TRUST BOUNDARY | The components, keys, records and assumptions on which a conclusion depends, including what an adversary or failure might alter.

Source ledger

Registry snapshot: Grounded-DI/grounded-di-replay-certificate-registry at commit 965328d1173fe7f46034c18efb53ed7aa2679d07, retrieved 2 November 2026. Each identifier below links to its primary public record at that commit; those records identify supporting reports and machine-readable evidence. Public package availability is as stated in each record.

[S1] Seven-Hash Replay Benchmark final local closure. Historical evidence ZIPs identified but not hosted in the repository.

[S2] Integrated Legal Evidence Chain Replay. Source, stage outputs, controls and reproducibility archive listed in the record.

[S3] NIST water temperature-dependent seven-boundary replay. Reproducibility package and project source listed in the record.

[S4] Single-Use Authorization and Crash Recovery v1. Browsable source, package, local certificate and checker listed in the record.

[S5] ReleaseGate V3 lost confirmation and recovery. Reproducibility package and separate clean-extraction recheck listed in the record.

[S6] ReleaseGate V4 authenticated receipt and planted-file control. Verification JSON, reproducibility package and extraction recheck listed in the record.

[S7] Delivery-to-inventory reconciliation replay. Reproducibility package and clean-extraction audit listed in the record.

[S8] VerdictBridge Demo 3 selective punitive-control replay. Specification, verifier, execution evidence and public package listed in the record.

The exhibit selection and interpretive framework are editorial. Source-reported results remain attributed to the project and their original boundaries.